Tecnológico Nacional de México

Instituto Tecnológico Superior de Felipe Carrillo Puerto

Ingeniería en Sistemas Computacionales

Asignatura
Graficación

Tema

EVIDENCIA DE APRENDIZAJE
Actividad 1.4 Evaluación Práctica

Profesor
Ing. Paloma Gongora Sabido

Alumno (s):
Benítez Constantino María Fernanda
Espinoza Balam Kary Yareana
Medina Tamayo Arturo Uriel

ISC - 3B

Felipe Carrillo Puerto a día 14 de Septiembre de 2026.



Guía práctica: Desarrollo gráfico

Processing

Introducción

Processing es un lenguaje de programación y un entorno orientado principalmente a la creación de gráficos, animaciones y aplicaciones interactivas. Su sintaxis permite trabajar de manera sencilla con formas geométricas, colores, posiciones y diferentes propiedades visuales.

Para esta actividad se utilizó Processing para representar gráficamente el Sistema Solar, utilizando figuras geométricas básicas y diferentes propiedades de dibujo. El ejercicio permitió identificar cómo se configura un lienzo, cómo se construyen figuras mediante código y cómo se modifican sus características.

Processing cuenta con una referencia oficial donde se documentan sus funciones, primitivas geométricas, propiedades de color y herramientas de dibujo.

2. Creación y configuración del lienzo

El primer paso consiste en establecer las dimensiones del espacio donde se realizará el dibujo.

La función principal es:

size(ancho, alto);

Por ejemplo:

void setup() {





```
  size(604, 600);
}
```

En este caso:

- 600 representa el **ancho** del lienzo en píxeles.
- 600 representa el **alto** del lienzo en píxeles.
- setup() se ejecuta una sola vez al iniciar el programa.
- size() establece las dimensiones de la ventana.

La documentación oficial indica que size() define el ancho y alto de la ventana en píxeles y que, cuando se utiliza setup(), debe colocarse como la primera instrucción dentro de esta función.

Aplicación en nuestro proyecto

Se utilizó:

```
void setup() {
  size(600, 600);
  smooth();
 textAlign(CENTER);
}
```

De esta manera se estableció el tamaño del lienzo y se configuró la apariencia del dibujo.





3. Dibujo de primitivas geométricas

Processing proporciona diferentes funciones para crear formas geométricas. Entre las principales se encuentran:

Primitiva	Sintaxis	Uso
Punto	point(x, y)	Dibuja un punto
Línea	line(x1, y1, x2, y2)	Dibuja una línea
Rectángulo	rect(x, y, ancho, alto)	Dibuja un rectángulo
Elipse	ellipse(x, y, ancho, alto)	Dibuja una elipse
Triángulo	triangle(x1,y1,x2,y2,x3,y3)	Dibuja un triángulo
Arco	arc()	Dibuja un arco

Processing incluye estas y otras primitivas dentro de su sistema de formas 2D.



Carretera Vigía Chico SN, C.P. 77200, Colonia Centro, Felipe Carrillo Puerto, Q. Roo
e-mail: direccion@itscarrillopuerto.edu.mx |tecnm.mx | carrillopuerto.tecnm.mx



Ejemplo

ellipse(200, 200, 100, 100);

La función ellipse() utiliza cuatro parámetros:

ellipse(x, y, ancho, alto)

Por defecto, los dos primeros valores indican la posición del centro y los otros dos determinan el ancho y alto de la elipse.

Aplicación en el Sistema Solar

Los planetas fueron representados utilizando elipses:

ellipse(329, 189, 40, 40);

Al utilizar el mismo valor para ancho y alto se obtiene un círculo.

Por ejemplo, Júpiter se representa mediante:

ellipse(329, 189, 40, 40);

Mientras que las órbitas se construyeron mediante pequeñas líneas colocadas alrededor de un centro común.

4. Aplicación de colores, estilos y formatos

Processing permite modificar la apariencia de las figuras mediante diferentes funciones.

Las principales utilizadas fueron:





background()

Permite establecer el color del fondo.

background(43, 11, 63);

En este proyecto se utilizó para crear el fondo morado.

fill()

Establece el color de relleno de una figura.

fill(255, 255, 0);
ellipse(12, 315, 205, 205);

En este caso se utiliza amarillo para representar el Sol.

stroke()

Establece el color del contorno.

stroke(190, 175, 200);

Se utilizó para las órbitas.

strokeWeight()

Permite modificar el grosor del contorno o de las líneas.

strokeWeight(2);





noStroke()

Elimina el contorno de las figuras.

noStroke();

noFill()

Elimina el relleno de una figura.

noFill();

Estas funciones forman parte de las herramientas de configuración visual de Processing.

5. Uso de colores RGB

Processing permite establecer colores mediante valores **RGB**:

fill(rojo, verde, azul);

Cada componente normalmente utiliza valores entre 0 y 255.

Por ejemplo:

fill(255, 0, 0);

produce rojo.

fill(0, 255, 0);

produce verde.





fill(0, 0, 255);

produce azul.

En el proyecto se utilizaron diferentes combinaciones RGB para distinguir los planetas.

Por ejemplo:

fill(39, 196, 110);

se utilizó para representar la Tierra.

6. Modificación de propiedades

Una de las ventajas de trabajar con gráficos mediante código es que las características de las figuras pueden modificarse directamente desde sus parámetros.

Posición

La posición se modifica mediante las coordenadas:

ellipse(270, 292, 25, 25);

Los valores 270 y 292 determinan la posición de la figura.

Tamaño

Los últimos valores determinan el tamaño:

ellipse(270, 292, 25, 25);





- 25 → ancho
- 25 → alto

Si cambiamos:

ellipse(270, 292, 50, 50);

el planeta aumenta de tamaño.

Contorno

El grosor puede modificarse con:

strokeWeight(5);

También podemos eliminarlo:

noStroke();

Relleno

Podemos cambiar el color mediante:

fill(255, 0, 0);

o eliminarlo:

noFill();





7. Aplicación práctica: Sistema Solar

Para demostrar los elementos anteriores se desarrolló una representación gráfica del Sistema Solar.

El proyecto utiliza:

- Un lienzo de 604 × 600 píxeles.
- Un fondo de color morado.
- El Sol mediante dos elipses.
- Los planetas mediante círculos.
- Las órbitas mediante líneas discontinuas.
- Diferentes colores para distinguir cada planeta.
- Texto para identificar los cuerpos celestes.
- Un contorno para los anillos de Saturno.

Un fragmento del código utilizado es:

```
// JÚPITER
fill(243, 156, 18);
ellipse(329, 189, 40, 40);

fill(255);
textSize(12);
text("Júpiter", 329, 219);
```





Se combinan **primitivas geométricas, color, posición, tamaño y texto** en un mismo elemento.

8. Procedimiento

Paso 1. Crear un nuevo proyecto en Processing.

Paso 2. Configurar el tamaño del lienzo mediante:

size(604, 600);

Paso 3. Establecer el color del fondo mediante:

background(43, 11, 63);

Paso 4. Crear las órbitas utilizando líneas y coordenadas.

Paso 5. Dibujar el Sol mediante dos elipses superpuestas.

Paso 6. Crear los planetas utilizando ellipse().

Paso 7. Aplicar diferentes colores mediante fill().

Paso 8. Modificar los contornos utilizando stroke(), strokeWeight() y noStroke().

Paso 9. Agregar los nombres de los planetas mediante text().

Paso 10. Ajustar las coordenadas y tamaños hasta obtener una composición similar a la imagen de referencia.





9. Ventajas de Processing

Durante la realización del ejercicio se pueden identificar las siguientes ventajas:

- **Sintaxis sencilla:** las funciones gráficas tienen nombres fáciles de relacionar con su función.
- **Facilidad para crear figuras:** permite dibujar círculos, elipses, líneas, rectángulos y otras formas mediante pocas instrucciones.
- **Control mediante código:** la posición, tamaño y color pueden modificarse directamente.
- **Orientación gráfica:** está diseñado específicamente para facilitar la creación de gráficos y animaciones.
- **Documentación:** cuenta con una referencia oficial que explica las funciones y proporciona ejemplos.
- **Animación:** la función draw() permite ejecutar continuamente un bloque de código, lo que facilita la creación de movimiento e interacción.

10. Limitaciones de Processing

También presenta algunas limitaciones dependiendo del objetivo:

- Para utilizar los gráficos directamente dentro de una página web, no es tan directo como trabajar con tecnologías web.

- La representación depende del entorno de Processing.
- Para proyectos web se requieren otras tecnologías o herramientas.
- Cuando el proyecto aumenta su complejidad, el código puede hacerse más extenso si no se organiza correctamente.





- La apariencia puede variar ligeramente respecto a una imagen de referencia debido al método de renderizado.

Requiere de instalación, el tiempo de espera es corto y es sencillo para comenzar a usar en el momento.



Carretera Vigía Chico SN, C.P. 77200, Colonia Centro, Felipe Carrillo Puerto, Q. Roo
e-mail: direccion@itscarrillopuerto.edu.mx |tecnm.mx | carrillopuerto.tecnm.mx



Código

```
void setup() {
  size(604, 600);
  smooth();
  textAlign(CENTER);
}

void draw() {

  // Fondo
  background(43, 11, 63);

  // Órbitas
  stroke(190, 175, 200);
  strokeWeight(1);

  // Centro de las órbitas

float cx = 12;
 float cy = 315;
```





```
// Orbitas
orbita(cx, cy, 140);
orbita(cx, cy, 180);
orbita(cx, cy, 220);
orbita(cx, cy, 260);
orbita(cx, cy, 300);
orbita(cx, cy, 350);
orbita(cx, cy, 400);
orbita(cx, cy, 450);
orbita(cx, cy, 500);

// Sol
noStroke();

// Borde naranja


fill(255, 165, 0);

ellipse(12, 315, 230, 230);

// Centro amarillo
fill(255, 255, 0);
```





```
  ellipse(12, 315, 205, 205);

  // Mercurio
  fill(190, 165, 150);
  ellipse(148, 257, 16, 16);

  fill(255);
  textSize(12);
  text("Mercurio", 148, 278);

  // Venus

fill(250, 170, 185);
  ellipse(192, 310, 20, 20);

fill(255);
  textSize(12);
  text("Venus", 192, 333);

  // Tierra
  fill(39, 196, 110);
```





```
  ellipse(270, 292, 25, 25);

  fill(255);
  textSize(12);
  text("Tierra", 270, 317);

  // Marte

fill(238, 80, 58);
  ellipse(231, 371, 19, 19);

  fill(255);
  textSize(12);

text("Marte", 231, 393);

  // Júpiter
  ellipse(329, 189, 40, 40);

  fill(255);
  textSize(12);
  text("Júpiter", 329, 219);
```





```
// Urano
fill(0, 255, 255);

ellipse(403, 212, 27, 27);

fill(255);
textSize(12);
text("Urano", 403, 237);

// Saturno
// Anillo
noFill();
stroke(225, 205, 120);
strokeWeight(2);

ellipse(361, 398, 50, 15);

// Planeta
noStroke();
fill(240, 230, 140);
ellipse(361, 398, 33, 33);
```





```
  fill(255);
  textSize(12);
  text("Saturno", 361, 426);

  // Neptuno
  fill(65, 105, 220);
  ellipse(439, 406, 27, 27);

  fill(255);
  textSize(12);
  text("Neptuno", 439, 431);
}

// Función para órbitas punteadas
void orbita(float x, float y, float radio) {

  noFill();



 stroke(190, 175, 200);
  strokeWeight(1);
```





```
    // Separación entre puntos
    float paso = 0.025;

    for (float a = 0; a < TWO_PI; a += paso) {

      // Punto de inicio

  float x1 = x + cos(a) * radio;
      float y1 = y + sin(a) * radio;

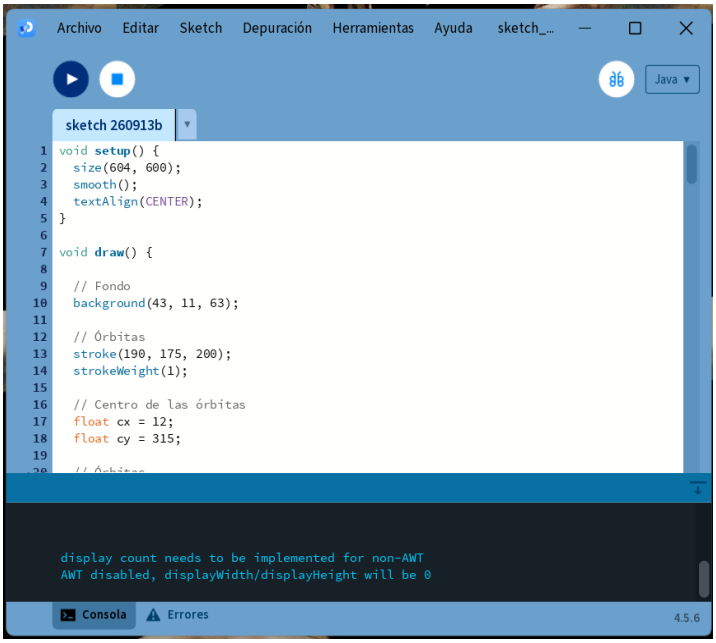
      // Punto final del pequeño segmento
      float x2 = x + cos(a + 0.010) * radio;
      float y2 = y + sin(a + 0.010) * radio;

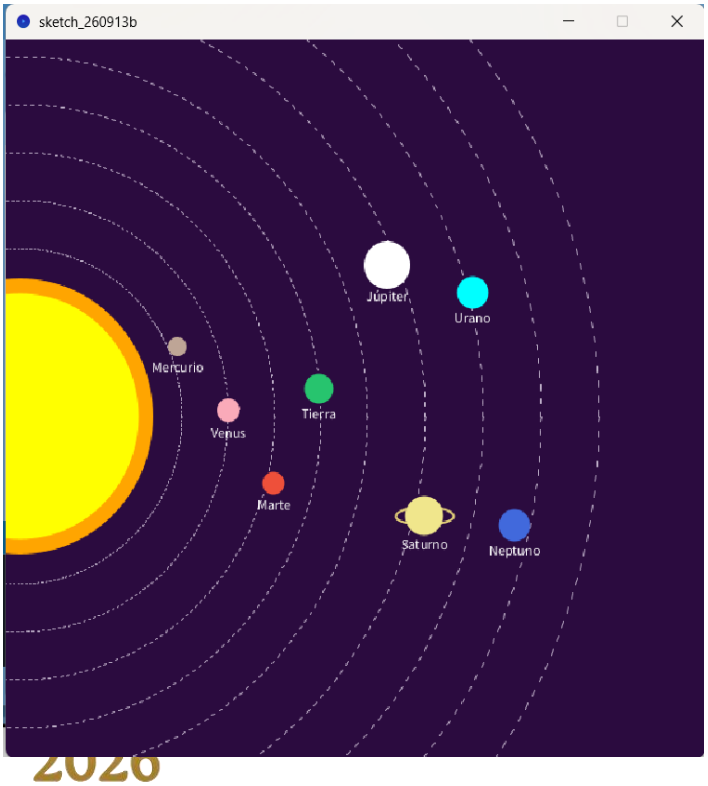

   line(x1, y1, x2, y2);
    }
  }
```



Carretera Vigía Chico SN, C.P. 77200, Colonia Centro, Felipe Carrillo Puerto, Q. Roo
e-mail: direccion@itscarrillopuerto.edu.mx |tecnm.mx | carrillopuerto.tecnm.mx

Evidencia


Archivo Editar Sketch Depuración Herramientas Ayuda sketch_...
Java
sketch 260913b
void setup() {
size(604, 600);
smooth();
textAlign(CENTER);
}
void draw() {
// Fondo
background(43, 11, 63);
// Órbitas
stroke(190, 175, 200);
strokeWeight(1);
// Centro de las órbitas
float cx = 12;
float cy = 315;
display count needs to be implemented for non-AWT
AWT disabled, displayWidth/displayHeight will be 0
Consola
Errores
4.5.6


sketch_260913b
Mercurio
Venus
Tierra
Marte
Júpiter
Saturno
Urano
Neptuno



Conclusión

El desarrollo del Sistema Solar permitió comprobar de manera práctica cómo Processing facilita la creación de gráficos mediante programación. A través de funciones como size(), ellipse(), fill(), stroke() y strokeWeight() fue posible controlar las dimensiones del lienzo, crear figuras geométricas, aplicar colores y modificar diferentes propiedades visuales.

Una de las principales ventajas observadas fue que los elementos gráficos pueden modificarse directamente desde el código, por lo que cambiar la posición, tamaño o color de un planeta solamente requiere modificar sus parámetros.

Además, Processing resulta adecuado para aprender los fundamentos de la programación gráfica, ya que sus funciones permiten relacionar fácilmente el código con el resultado visual. La función draw() también permite ampliar posteriormente el proyecto para agregar animaciones o interacción.





Referencias

Processing Foundation. (s. f.). *ellipse()*. Processing Reference. https://processing.org/reference/ellipse_.html

Processing Foundation. (s. f.). *Getting started*. Processing. https://processing.org/tutorials/gettingstarted/

Processing Foundation. (s. f.). *Overview*. Processing. https://processing.org/tutorials/overview/

Processing Foundation. (s. f.). *Reference*. Processing. https://processing.org/reference/

Processing Foundation. (s. f.). *setup()*. Processing Reference. https://processing.org/reference/setup_.html

Processing Foundation. (s. f.). *size()*. Processing Reference. https://processing.org/reference/size_.html





HTML

¿Es necesario instalar algún programa?

Como es HTML es un lenguaje nativo que los navegadores web como (Google Chrome, Edge, Firefox) ya entienden por defecto. Solo necesitamos escribirlo en un archivo de texto con extensión .html. (aunque en este caso yo lo hice en el OneCompiler

¿Necesita librerías navegador editor o archivos adicionales?

Pero si necesita un editor de código por ejemplo (Visual Studio Code, Bloc de notas o plataformas online como OneCompiler) y se usa el navegador web para abrir y ver el resultado

Y para las figuras en 2D no se instalan librerías externas. Los dibujos de las figuras se realizan con la API nativa de Canvas en JavaScript. Si en otro proyecto quisiera usar una librería externa (como Three.js o p5.js), no se descargan instaladores, sino que se importan agregando una sola línea en el HTML dentro del <head> o <script> que son los contenedores donde hice cada lienzo cada planeta cada orbita

1. ¿Como se crea el proyecto?

El proyecto se crea simplemente abriendo un bloc de notas o editor de texto, escribiendo el código y guardándolo con el nombre de extensión (index.html.)

¿Dónde se ejecuta? Lo puedo ejecutar directo dentro de cualquier navegador web (Google Chrome, Microsoft Edge, Safari o Firefox). Solo le das doble clic al archivo index.html o usas un compilador en línea como (OneCompiler) que es el que use en este caso

¿Cómo se configura el área de dibujo o "lienzo"?





Con la parte especifica del código

- <canvas id="c" width="600" height="600"></canvas>

Esto sirve para dar el tamaño de 600 de ancho el primero y 600 de largo el segundo y tiene sus canvas de apertura y cierre esto sirve para cada elemento en especifico y no afecte a los demas

2. ¿Qué tan fácil es escribir y entender el código?

Es medianamente sencillo por que funciona similar a P5 al usar palabras en ingles como “arc” para los arcos o “rect” para los rectángulos con lo que hice el código y además para instalar los colores se coloca junto a la misma linea del código con un # y el código del color ejemplo es que #FFF es color blanco
¿Cuántas instrucciones se necesitan para hacer algo sencillo? Me lleva entren 2 a 4 lineas por figura porque en una iría la figura el color los tamaños y en otra iría las coordenas donde coloco la figura y sus posiciones izquierda derecha y que tan arriba o abajo la quiero

3. ¿Cómo se crean rectángulos, círculos, líneas, etc.?

Se crean con dos líneas principales como ejemplo tomare un rectángulo que es el del fondo del espacio

```
x.fillStyle = '#260940'; x.fillRect(0, 0, 600, 600);
```

esta es una variable donde podremos guardar otras figuras siempre y cuando coloquemos que va dentro de la variable “X” el # del color y colocamos que la variable X es un rectángulo y los 0,0 son para colocarlo en medio del código y 600 de ancho y 600 de largo y como otro ejemplo tomo mis 2 lineas de código que hacen las orbitas con arcos

```
x.strokeStyle = '#a682c4'; x.setLineDash([4, 6]);
```





```
[130, 170, 215, 265, 320, 390, 450, 500].forEach(r => {
```

La primera liena le indica que será una linea recta el # el color y le indicamos que se guardará dentro de la variable x con 4 pixeles blancos dibujados y 6 pixeles de separación .
Las segundas son las orbitas que estoy haciendo y la coloco en cada coordenada si borro un numero el dibujo de la linea desaparece

¿Qué tan fácil es modificar posición, tamaño y contorno? Dependiendo de la figura las más fáciles para mí son los rectángulos, líneas y círculos y las más difíciles son los triángulos por que por ejemplo modificar un rectángulo solo tiene dos puntos que son x1 y x2 que el x1 es el ancho y el x2 el largo y es bastante sencillo y la segunda linea donde va las coordenadas donde indico que tan arriba o abajo quiero que este y que tan a la izquierda o a la derecha quiero que este

4. ¿Cómo se aplican colores, rellenos, bordes, transparencia y otros efectos?

Esto es fácil para aplicar color solo colocamos la figura y luego un # con el código del color que vayamos a usar en mi caso yo saque los colores de esta pagina llamada (https://htmlcolorcodes.com/es/) los bordes se pueden añadir con la figura la cual queremos el borde y el color código del color del borde x.strokeStyle = '#a682c4'

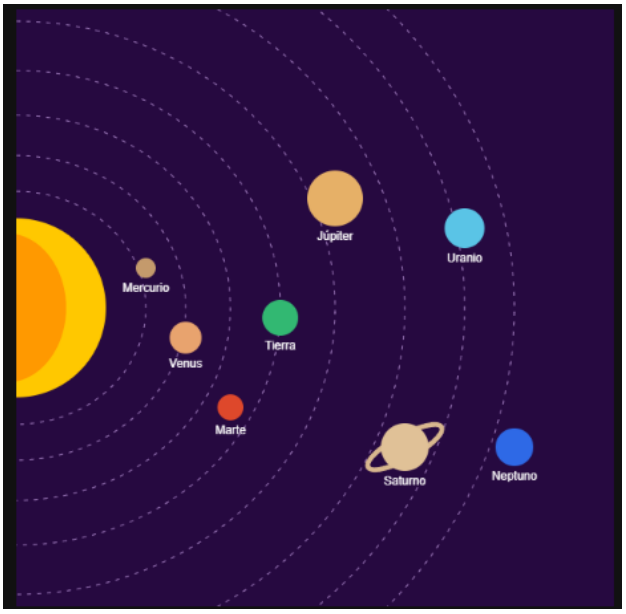
¿Qué posibilidades y limitaciones tiene?
Una de las posibilidades que le veo es que a las figuras 2D se le pueden colocar efectos que hacen que los cuadros se vean estéticamente mejor como sombreado transparencia sin necesidad de instalar librerías aparte esto por que el navegador ya las tiene y una limitación

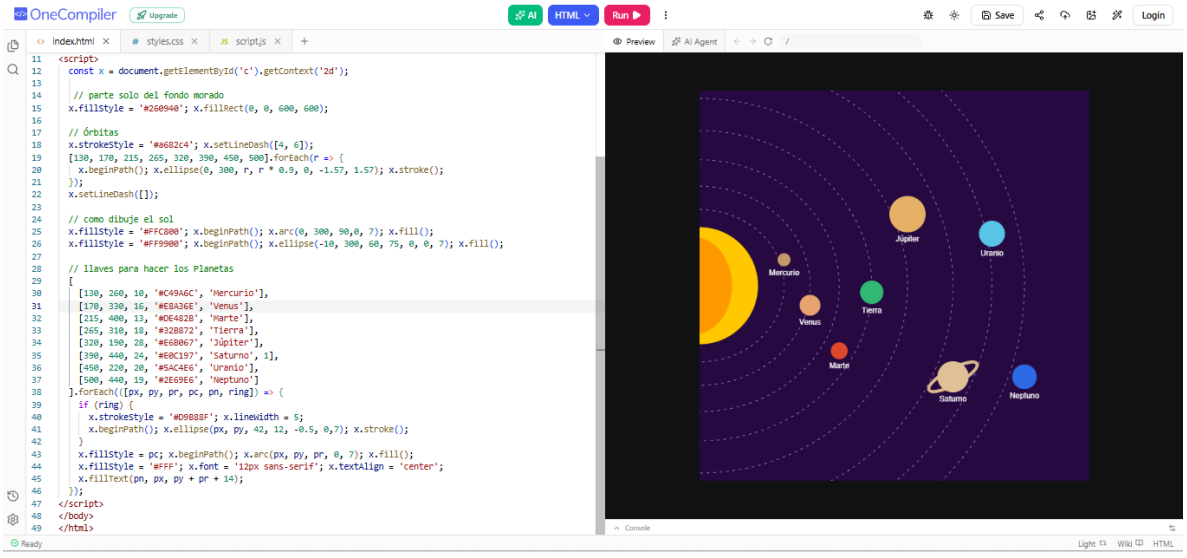




Es que cuando hago algo en el canvas, el navegador se olvida qué era (se convierten en píxeles planos). Si quiero mover un planeta, debo borrar todo el lienzo y volver a hacer la linea de código completa.

Evidencia


Júpiter
Uranio
Mercurio
Tierra
Venus
Marte
Saturno
Neptuno


OneCompiler
Upgrade
AI
HTML
Run
Save
Login
index.html
styles.css
script.js
Preview
AI Agent
<script>
const x = document.getElementById('c').getContext('2d');
// parte solo del fondo morado
x.fillStyle = '#260940'; x.fillRect(0, 0, 600, 600);
// Órbitas
x.strokeStyle = '#a682c4'; x.setLineDash([4, 6]);
[130, 170, 215, 265, 320, 390, 450, 500].forEach(r => {
x.beginPath(); x.ellipse(0, 300, r, r * 0.9, 0, -1.57, 1.57); x.stroke();
});
x.setLineDash([]);
// como dibuje el sol
x.fillStyle = '#FFC800'; x.beginPath(); x.arc(0, 300, 90,0, 7); x.fill();
x.fillStyle = '#FF9900'; x.beginPath(); x.ellipse(-10, 300, 60, 75, 0, 0, 7); x.fill();
// llaves para hacer los Planetas
[
[130, 260, 10, '#C49A6C', 'Mercurio'],
[170, 330, 16, '#EBA36E', 'Venus'],
[215, 400, 13, '#DE482B', 'Marte'],
[265, 310, 18, '#32B872', 'Tierra'],
[320, 190, 28, '#E6B067', 'Júpiter'],
[390, 440, 24, '#E0C197', 'Saturno', 1],
[450, 220, 20, '#5AC4E6', 'Uranio'],
[500, 440, 19, '#2E69E6', 'Neptuno']
].forEach(([px, py, pr, pc, pn, ring]) => {
if (ring) {
x.strokeStyle = '#D9B88F'; x.lineWidth = 5;
x.beginPath(); x.ellipse(px, py, 42, 12, -0.5, 0,7); x.stroke();
}
x.fillStyle = pc; x.beginPath(); x.arc(px, py, pr, 0, 7); x.fill();
x.fillStyle = '#FFF'; x.font = '12px sans-serif'; x.textAlign = 'center';
x.fillText(pn, px, py + pr + 14);
});
</script>
</body>
</html>
Console
Ready
Light
Wiki
HTML



Código

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Sistema Solar</title>

  <style>body{display:grid;place-
items:center;height:100vh;background:#111;margin:0 }</style>
</head>
<body>
<canvas id="c" width="600" height="600"></canvas>
<script>
  const x = document.getElementById('c').getContext('2d');

   // parte solo del fondo morado
  x.fillStyle = '#260940'; x.fillRect(0, 0, 600, 600);

  // Órbitas
  x.strokeStyle = '#a682c4'; x.setLineDash([4, 6]);
  [130, 170, 215, 265, 320, 390, 450, 500].forEach(r => {
    x.beginPath(); x.ellipse(0, 300, r, r * 0.9, 0, -1.57, 1.57);
x.stroke();
  });
  x.setLineDash([]);

  // como dibuje el sol
  x.fillStyle = '#FFC800'; x.beginPath(); x.arc(0, 300, 90,0, 7); x.fill();
  x.fillStyle = '#FF9900'; x.beginPath(); x.ellipse(-10, 300, 60, 75, 0, 0,
7); x.fill();

  // llaves para hacer los Planetas
  [
    [130, 260, 10, '#C49A6C', 'Mercurio'],
```





```
 [170, 330, 16, '#E8A36E', 'Venus'],
    [215, 400, 13, '#DE482B', 'Marte'],
    [265, 310, 18, '#32B872', 'Tierra'],
    [320, 190, 28, '#E6B067', 'Júpiter'],
    [390, 440, 24, '#E0C197', 'Saturno', 1],
    [450, 220, 20, '#5AC4E6', 'Uranio'],
    [500, 440, 19, '#2E69E6', 'Neptuno']
  ].forEach(([px, py, pr, pc, pn, ring]) => {
    if (ring) {
      x.strokeStyle = '#D9B88F'; x.lineWidth = 5;
      x.beginPath(); x.ellipse(px, py, 42, 12, -0.5, 0,7); x.stroke();
    }
    x.fillStyle = pc; x.beginPath(); x.arc(px, py, pr, 0, 7); x.fill();
    x.fillStyle = '#FFF'; x.font = '12px sans-serif'; x.textAlign =
'center';
    x.fillText(pn, px, py + pr + 14);
  });
</script>
</body>
</html>
```



Carretera Vigía Chico SN, C.P. 77200, Colonia Centro, Felipe Carrillo Puerto, Q. Roo
e-mail: direccion@itscarrillopuerto.edu.mx |tecnm.mx | carrillopuerto.tecnm.mx



Conclusión

Para mi esta practica fue demasiado útil para poder aprender a graficar y usar el lenguaje HTML ya que es un lengua que requiere de una estructura de pasos y tuve que aprender cada uno de ellos a la hora de poner cada figura en su parte del código debido a que como funcionan los canvas y bodys si al momento de abrir no cierras cada uno estos no sabran cuando dejar de ejecutarse lo que hace que el código no resulte como queríamos a lo cual cada canvas nos ayuda en esa parte del código sin contar que también tuve que aprender a unir las formas con sus coordenadas y sus colores para que diera la forma posición y color que quería es interesante porque en cierto modo cada parte funciona y es lo que lo hace un lenguaje en parte fácil de aprender pero tedioso de programar en mi opinión



Referencias

DropCoding. (s. f.). *Cómo crear formas y figuras geométricas en CSS*. https://dropcoding.com/figuras-geometricas-css/

HTML Color Codes. (s. f.). *Códigos de colores HTML*. https://htmlcolorcodes.com/es/

Mozilla Developer Network. (s. f.). *Dibujando formas con canvas*. MDN Web Docs. https://developer.mozilla.org/es/docs/Web/API/Canvas_API/Tutorial/Drawing_shapes

YouTube. (s. f.). *Video de YouTube* [Video]. https://www.youtube.com/watch?v=8TxbgTGrK64

YouTube. (s. f.). *Video de YouTube* [Video]. https://www.youtube.com/watch?v=jAacyafiHmo





CSS

Introducción: Desarrollo de lienzo con CSS

El diseño del CSS posibilita establecer una **separación entre el contenido y la forma de presentación del documento** (dada por las fuentes, los colores y las capas empleadas). Así se puede lograr que muchos documentos HTML compartan la apariencia, utilizando una única hoja de estilo para todos (que se especifica en un archivo **.css**). Gracias a esta particularidad, se evita tener que repetir el código en la estructura.

OBJETIVO: Representar el Sistema Solar mediante HTML5 y CSS3 dentro de Visual Studio Code, utilizando un fondo morado, órbitas circulares punteadas, planetas con colores sólidos y sus respectivos nombres.

EQUIPO, MATERIAL Y RECURSOS

- Computadora
- Conexión a internet
- Navegador web (Google Chrome o Microsoft Edge)
- Editor de código Visual Studio Code





DESARROLLO

1. Instalación y requisitos: No hizo falta instalar programas extra ni librerías. Solo se usó Visual Studio Code para escribir los archivos index.html y styles.css, y el navegador web para ver el resultado.

2. Configuración y entorno de trabajo: Se creó un recuadro de 600x600 píxeles con fondo morado (#2b0b3f). Se usó la propiedad position: relative para que sirva como un lienzo donde se pueden acomodar los elementos con coordenadas exactas.

3. Cómo se calculó cada elemento:

- El Sol: Se calculó como un círculo de 200 píxeles colocado a la izquierda con left: -100px para que solo se vea la mitad dentro del lienzo. Se le puso un borde grueso de 15 píxeles en color naranja para simular su capa externa.
- Las Órbitas: Se calcularon como círculos exactos centrados en el Sol. Para que fueran concéntricas (una dentro de otra), se aumentó el tamaño de 80 en 80 píxeles empezando desde los 300px (Mercurio con 300px, Venus con 380px, Tierra con 460px, hasta llegar a Neptuno con 1060px). Se usó border: 1px dashed white para hacer el efecto de línea punteada.
- Los Planetas (Tamaños y Colores): Se ajustaron los tamaños según el planeta de forma proporcional. Mercurio es el más pequeño (15px), la Tierra y Venus medianos (20px-24px), Júpiter el más grande (40px) y





Neptuno/Urano medianos-grandes (26px). A cada uno se le asignó un color directo (rojo para Marte, verde para la Tierra, naranja para Júpiter, etc.).

- Ubicación de los Planetas:

  Para que cada planeta quedara montado exactamente sobre su línea de órbita, se calculó su distancia desde la izquierda (left) y desde arriba (top) usando prueba y error hasta alinearlos en diagonal y curva.

- Anillo de Saturno: Se calculó una elipse aplastada de 48px de ancho por 10px de alto, colocada sobre el planeta Saturno y rotada -20 grados (rotate(-20deg)) para darle el efecto inclinado.

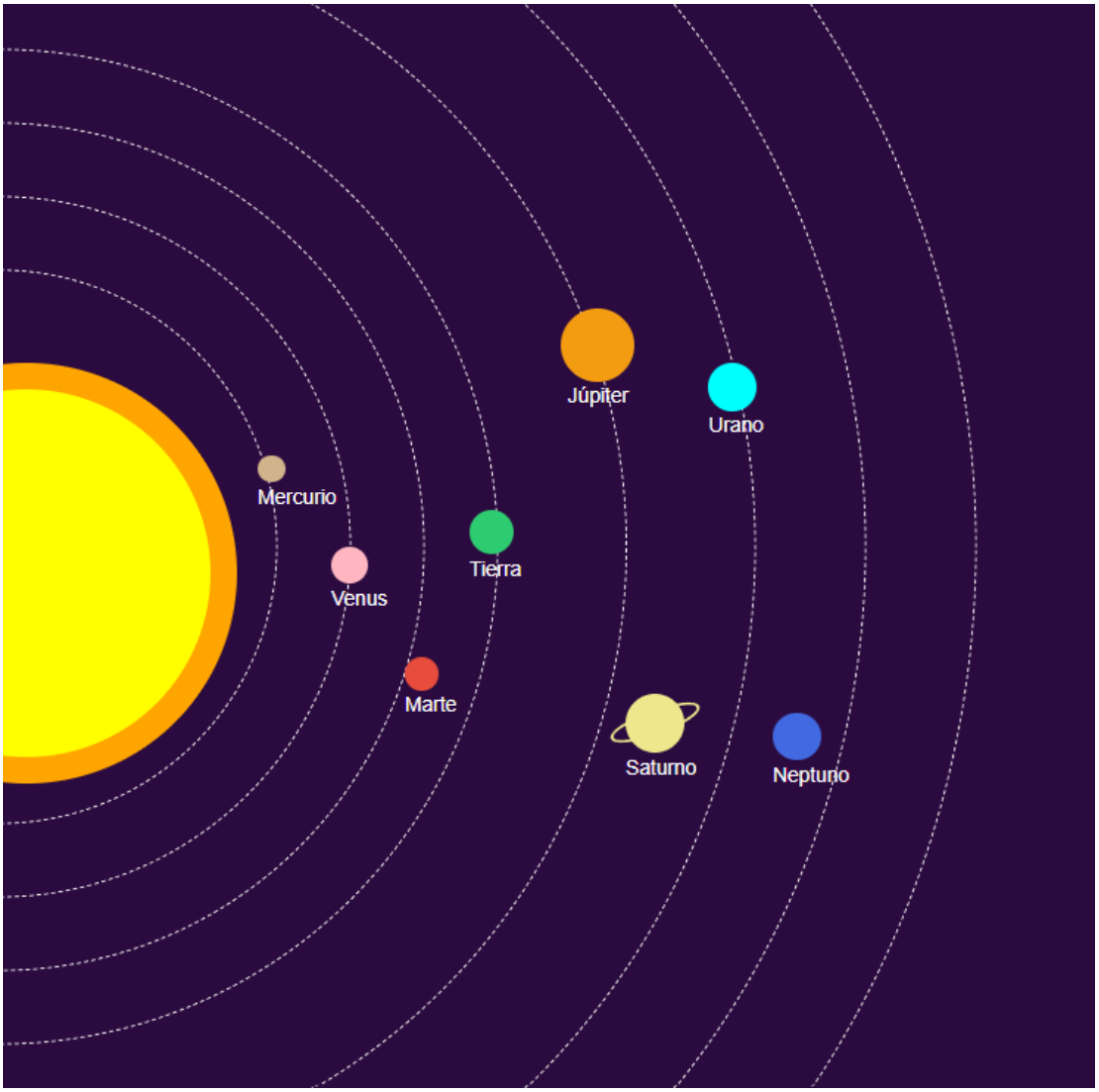
4. Sintaxis y facilidad de uso: El código es muy sencillo de entender. Para cada planeta solo se usó la propiedad border-radius: 50% para hacerlo redondo y position: absolute con top y left para moverlo a su lugar.

5. Estilos y nombres: Debajo de cada planeta se colocó un texto pequeño en color blanco con su nombre (*Mercurio, Venus, Tierra, Marte, Júpiter, Saturno, Urano y Neptuno*), centrado con text-align: center.

6. Posibilidades y limitaciones: Usar CSS es una forma rápida y ligera de hacer dibujos para páginas web, aunque la principal dificultad fue calcular a ojo las posiciones top y left para que los planetas tocaran exactamente sus líneas de órbita.



Evidencia


Júpiter
Urano
Mercurio
Tierra
Venus
Marte
Saturno
Neptuno



Ventajas de CSS

- Con CSS no tienes que describir repetidamente cómo se ven los elementos individuales. Esto ahorra tiempo, hace el código más corto y menos propenso a errores.

- Te permite tener múltiples estilos en una página HTML, y esto hace que las posibilidades de personalización sean casi infinitas. Hoy en día, esto se está volviendo una necesidad más que algo básico.

- **Accesibilidad:** CSS ofrece herramientas y técnicas que permiten mejorar la accesibilidad de un sitio web para usuarios con discapacidades visuales o cognitivas. Esto incluye la posibilidad de cambiar el tamaño del texto, ajustar el contraste y utilizar técnicas de diseño responsivo para adaptarse a diferentes dispositivos y tamaños de pantalla.

- **Eficiencia:** Al utilizar CSS, los estilos se pueden aplicar a múltiples elementos a la vez mediante clases o selectores, lo que reduce la cantidad de código necesario y mejora la eficiencia del diseño. Esto resulta en tiempos de carga más rápidos y una mejor experiencia de usuario.





Limitaciones

- **Limitaciones en el diseño visual:** A pesar de sus capacidades visuales, CSS tiene ciertas limitaciones en términos de diseño. Por ejemplo, no es posible crear formas complejas, como círculos o triángulos, utilizando solo CSS. Para lograr esto, es necesario utilizar imágenes o técnicas más avanzadas como SVG (Scalable Vector Graphics).

- **Compatibilidad del navegador:** Uno de los mayores desafíos de CSS es la compatibilidad con diferentes navegadores. Cada navegador puede interpretar el código CSS de manera ligeramente diferente, lo que puede resultar en diferencias visuales entre ellos. Además, versiones antiguas de navegadores pueden no ser compatibles con ciertas funcionalidades más nuevas de CSS.
- **Limitaciones en la animación:** CSS ofrece capacidades básicas de animación, como transiciones y transformaciones. Sin embargo, para animaciones más complejas, puede ser necesario recurrir a bibliotecas y frameworks adicionales, como JavaScript o CSS preprocessors.





Conclusión

En conclusión, el desarrollo de esta práctica permitió replicar de forma clara y precisa el Sistema Solar utilizando únicamente herramientas web nativas como HTML5 y CSS3. Mediante el uso de coordenadas absolutas, bordes redondeados y arcos concéntricos, se logró estructurar un diseño limpio con un lienzo morado donde cada cuerpo celeste se encuentra ubicado sobre su órbita correspondiente. Esta actividad demostró que el uso de CSS3 es una alternativa eficiente y ligera para crear ilustraciones vectoriales directamente en código sin depender de imágenes externas, siendo su principal reto la calibración manual de los tamaños y coordenadas para lograr el acomodo visual exacto de cada elemento.





Código

```
body {
  background-color: #111;
  font-family: sans-serif;
}

/* Lienzoo morado */
.espacio {
  width: 600px;
  height: 600px;
  background-color: #2b0b3f;
  position: relative;
  margin: 20px auto;
  overflow: hidden;
}


/* Sol en el borde izquierdo */
.sol {
  width: 200px;
  height: 200px;
  background-color: yellow;
  border: 15px solid orange;
  border-radius: 50%;
  position: absolute;
  top: 200px;
  left: -100px;
}

/* Estilo general para las órbitas */
.orbita {
  border: 1px dashed white;
  border-radius: 50%;
  position: absolute;
  top: 300px;
  left: 0px;
  transform: translate(-50%, -50%);
}
```





```
/* Tamaños de cada órbita */
.o1 { width: 300px; height: 300px; }
.o2 { width: 380px; height: 380px; }
.o3 { width: 460px; height: 460px; }
.o4 { width: 540px; height: 540px; }
.o5 { width: 680px; height: 680px; }
.o6 { width: 820px; height: 820px; }
.o7 { width: 940px; height: 940px; }
.o8 { width: 1060px; height: 1060px; }

/* Configuración de los nombres y planetas */
.planeta {


  position: absolute;
  text-align: center;
}

.planeta p {
  color: white;
  font-size: 11px;
  margin: 2px 0 0 0;


}

/* Colores y posiciones de cada planeta */
.p-mercurio { top: 250px; left: 140px; }
.circulo-mercurio {
  width: 15px;
  height: 15px;
  background-color: #d2b48c;
  border-radius: 50%;
}

.p-venus { top: 300px; left: 180px; }
.circulo-venus {
  width: 20px;
  height: 20px;
  background-color: #ffb6c1;
  border-radius: 50%;
}
```





```
.p-tierra { top: 280px; left: 255px; }
.circulo-tierra {


 width: 24px;
  height: 24px;
  background-color: #2ecc71;
  border-radius: 50%;
}

.p-marte { top: 360px; left: 220px; }
.circulo-marte {
  width: 18px;
  height: 18px;
  background-color: #e74c3c;
  border-radius: 50%;
}

.p-jupiter { top: 170px; left: 305px; }
.circulo-jupiter {
  width: 40px;
  height: 40px;
  background-color: #f39c12;
  border-radius: 50%;
}


.p-saturno { top: 380px; left: 340px; }
.circulo-saturno {
  width: 32px;
  height: 32px;
  background-color: #f0e68c;
  border-radius: 50%;
  position: relative;


}

.anillo {
  width: 48px;
  height: 10px;
  border: 2px solid #f0e68c;
```



Carretera Vigía Chico SN, C.P. 77200, Colonia Centro, Felipe Carrillo Puerto, Q. Roo
e-mail: direccion@itscarrillopuerto.edu.mx |tecnm.mx | carrillopuerto.tecnm.mx



```
  border-radius: 50%;
   position: absolute;
   top: 9px;
   left: -10px;
   transform: rotate(-20deg);
}

.p-urano { top: 200px; left: 385px; }
.circulo-urano {
   width: 26px;
   height: 26px;
   background-color: #00ffff;
   border-radius: 50%;
}

.p-neptuno { top: 390px; left: 420px; }
.circulo-neptuno {
   width: 26px;
   height: 26px;
   background-color: #4169e1;
   border-radius: 50%;
}
```





Código: index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Sistema Solar</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>

  <div class="espacio">
    <!-- Sol -->
    <div class="sol"></div>

    <!-- Órbitas -->
    <div class="orbita o1"></div>
    <div class="orbita o2"></div>
    <div class="orbita o3"></div>
    <div class="orbita o4"></div>
    <div class="orbita o5"></div>
    <div class="orbita o6"></div>
    <div class="orbita o7"></div>
    <div class="orbita o8"></div>

    <!-- Planetas y sus nombres -->
    <div class="planeta p-mercurio">
      <div class="circulo-mercurio"></div>
      <p>Mercurio</p>
    </div>

    <div class="planeta p-venus">
      <div class="circulo-venus"></div>
      <p>Venus</p>
    </div>

    <div class="planeta p-tierra">
      <div class="circulo-tierra"></div>
```





```
        <p>Tierra</p>
    </div>

    <div class="planeta p-marte">
      <div class="circulo-marte"></div>
      <p>Marte</p>
    </div>

    <div class="planeta p-jupiter">
      <div class="circulo-jupiter"></div>
      <p>Júpiter</p>
    </div>

    <div class="planeta p-saturno">
      <div class="circulo-saturno">
        <div class="anillo"></div>
      </div>
      <p>Saturno</p>
    </div>

    <div class="planeta p-urano">
      <div class="circulo-urano"></div>
      <p>Urano</p>
    </div>

    <div class="planeta p-neptuno">
      <div class="circulo-neptuno"></div>
      <p>Neptuno</p>
    </div>
  </div>

</body>
</html>
```





Referencias

Definición.de. (s. f.). *CSS: Qué es, ventajas, definición y concepto*. https://definicion.de/css/

Medium Multimedia. (s. f.). *Desventajas de CSS: ¿Limitaciones a considerar en el desarrollo web?* https://www.mediummultimedia.com/web/cuales-son-las-desventajas-de-css/

